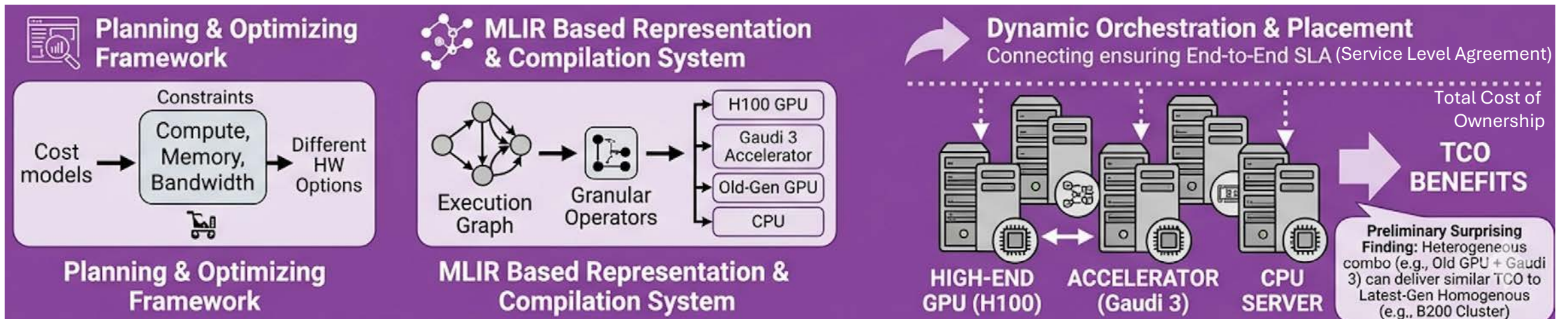
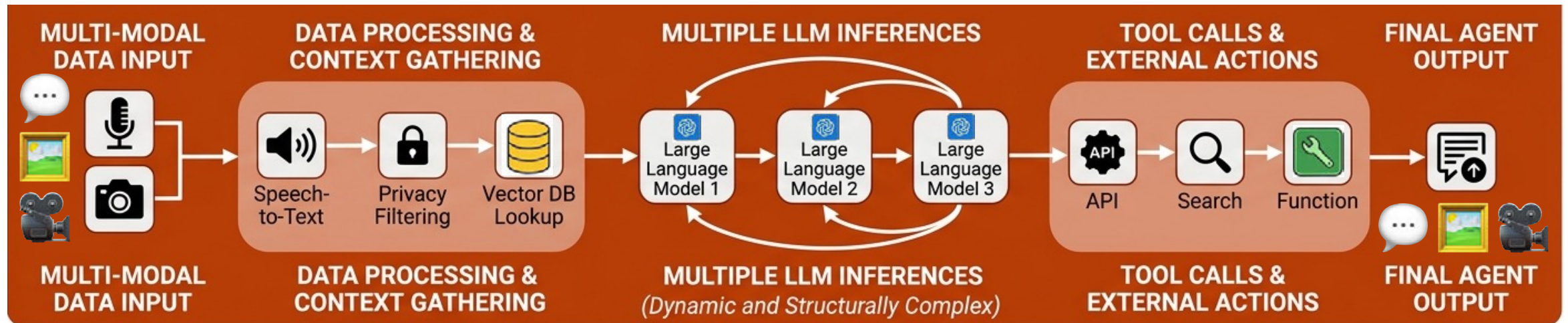


Efficient & Scalable *Agentic* AI

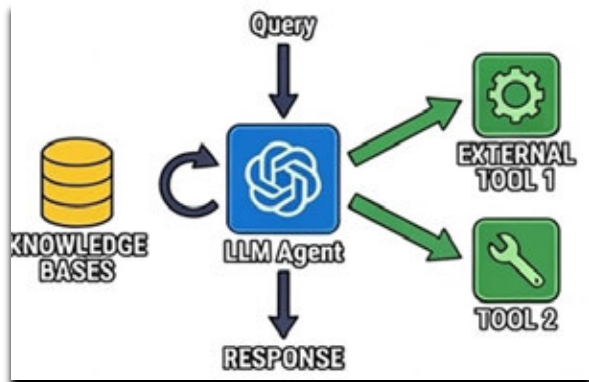


with *Heterogeneous* Systems

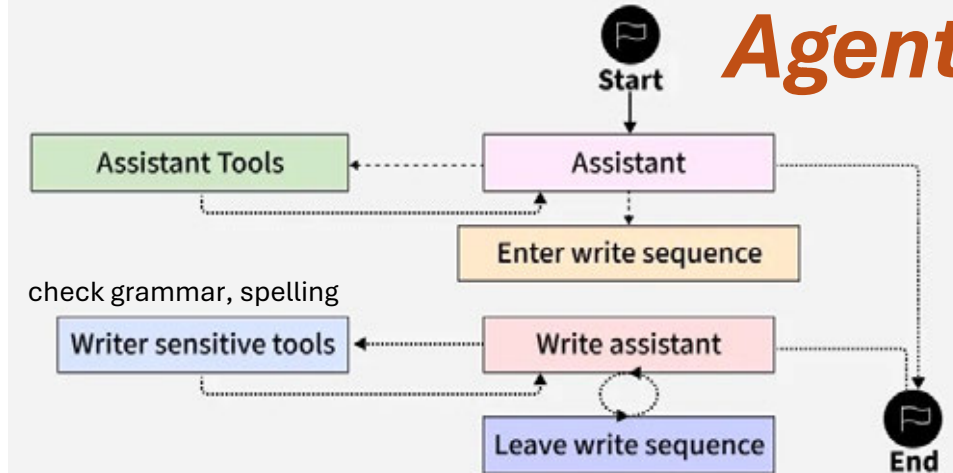




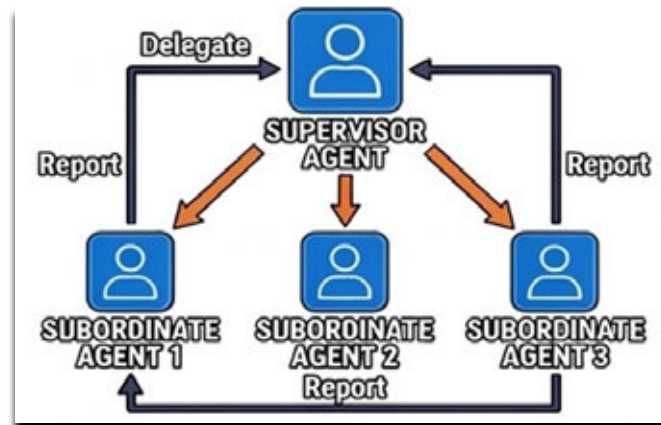
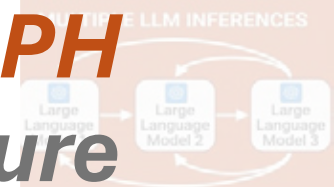
LangGraph's taxonomy



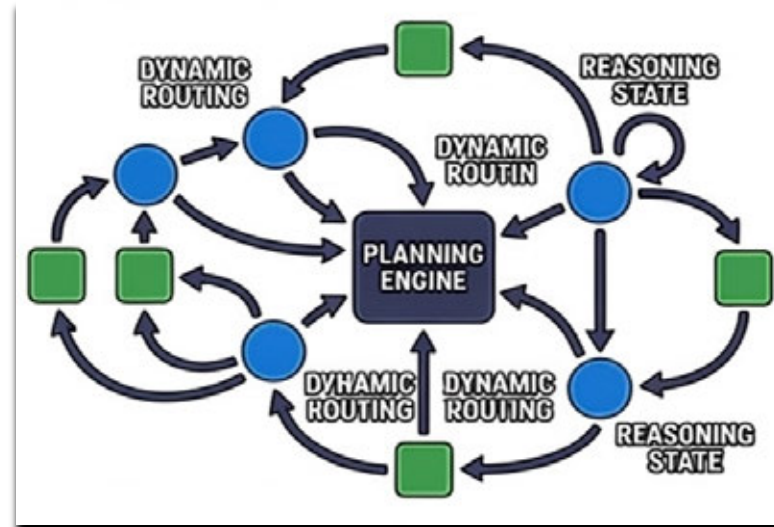
relationship with tools



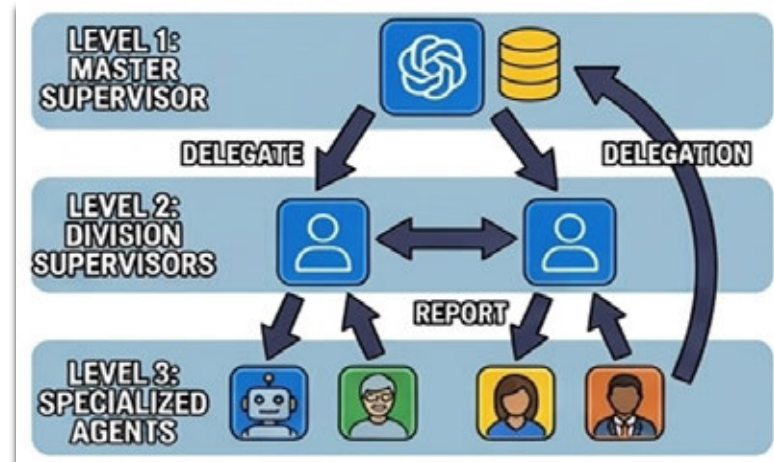
Agent Compute GRAPH Architecture



interconnected topology



scalable & general schemes



Single node vs. KV-cache disaggregation

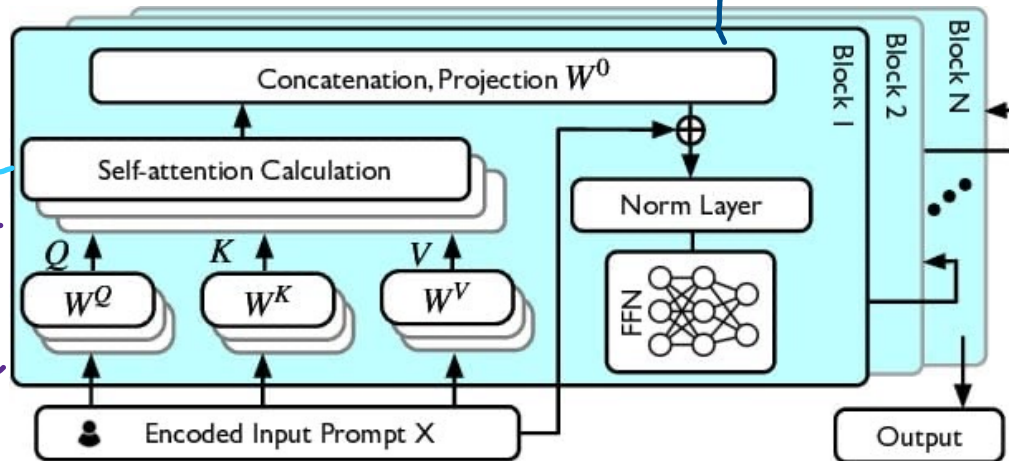
Autoregressive LLM

$$H_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$$

$$\text{Multi-Head Attention} = \text{Concat}(H_1, H_2, \dots, H_h)W^O$$

Observation: $Q_{\text{new token}}$ will be discarded after use.
Past **K & V** are static **but calculated repeatedly**.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

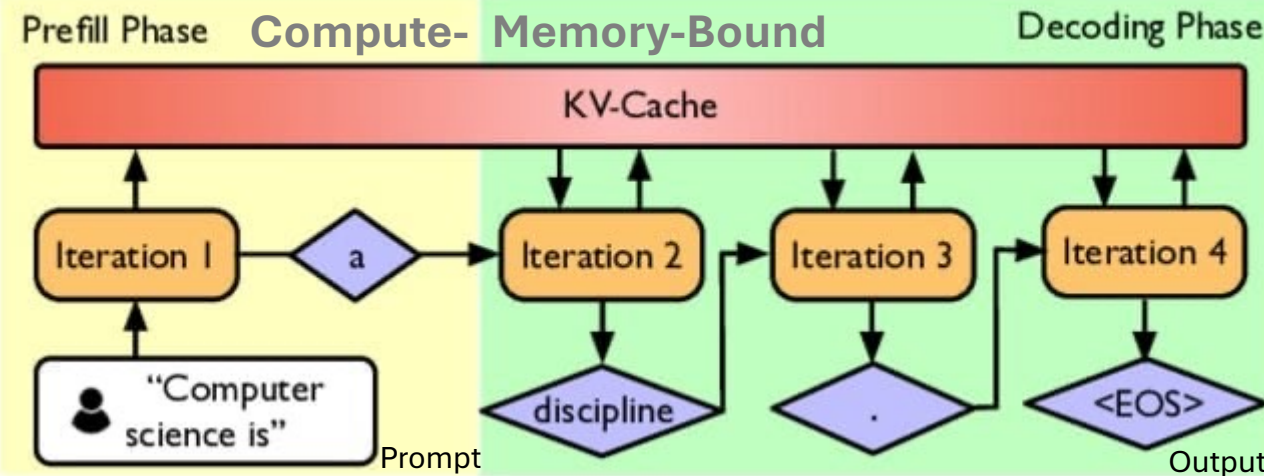
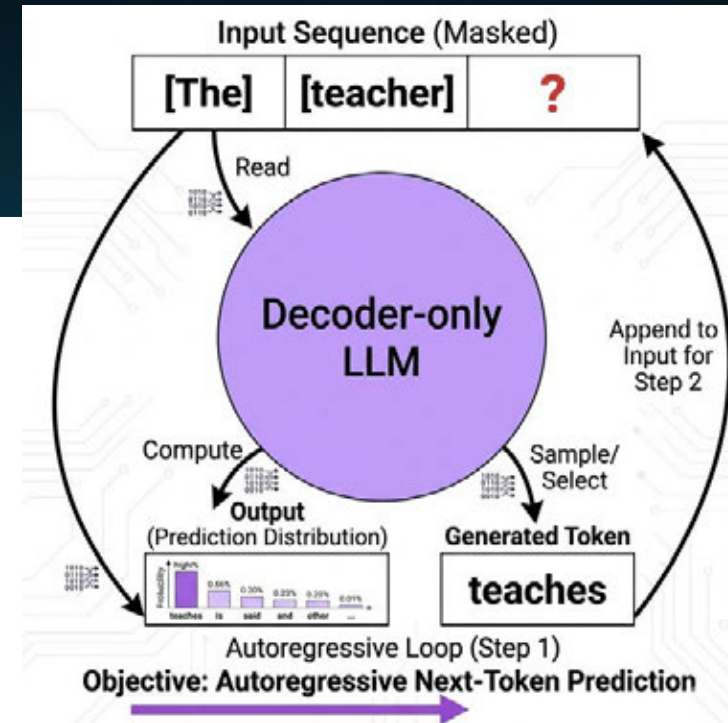


$$Q = XW^Q; K = XW^K; V = XW^V$$

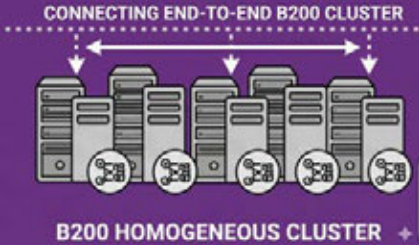
KV CACHE (Memory)

Token	Key Vector	Value Vector
[The]	[K1]	[V1]
[teacher]	[K2]	[V2]

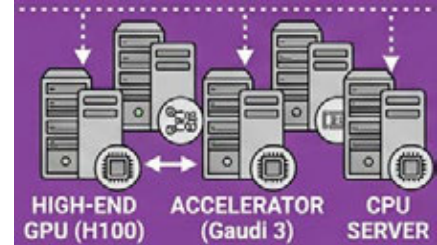
Write from prefill
Read from decode



HBM → DRAM → SSD trade-off: latency ↑, but capacity ↑.



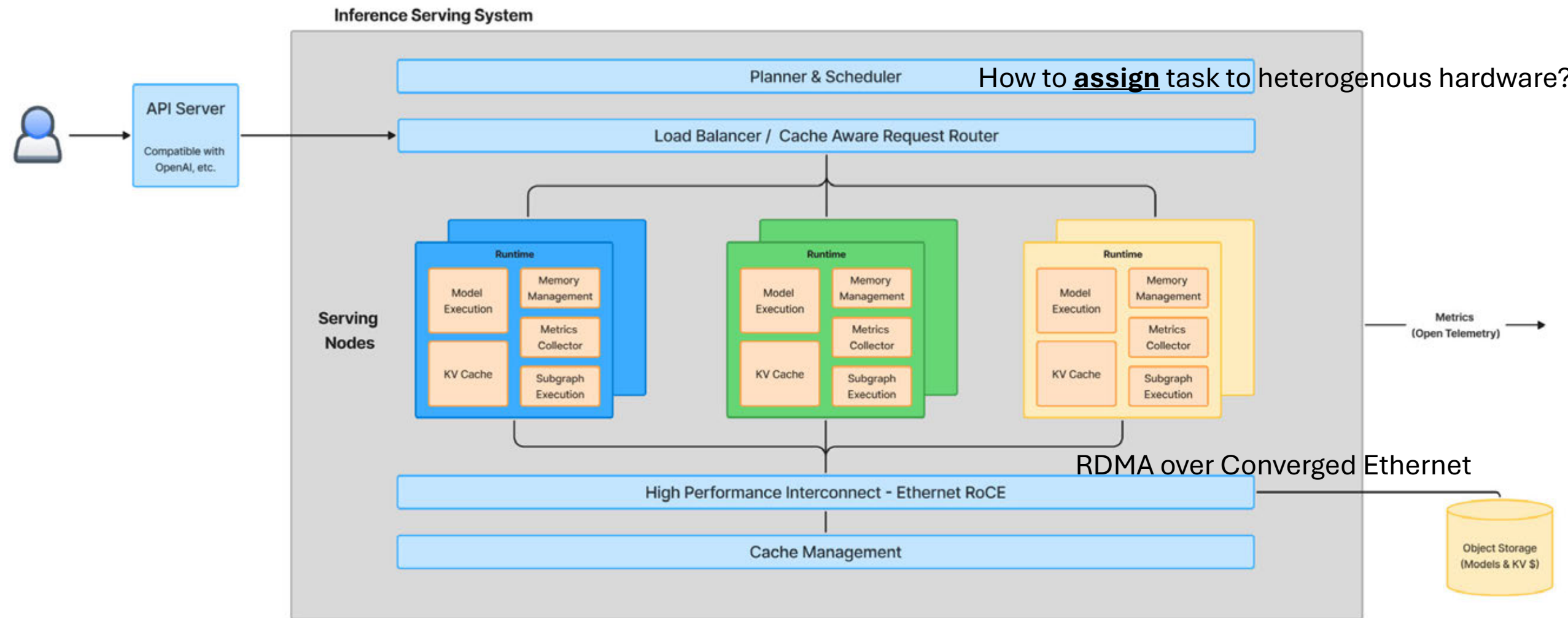
Homogeneous vs. Heterogeneous Systems



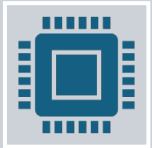
	Homogeneous	Heterogeneous
System Architecture	identical nodes + same hardware config.	different compute, memory, and network.
Examples	<i>NVIDIA GB200 NVL72</i>	<i>GPUs + CPUs + memory-rich nodes + specialized accelerators</i>
Resource	<u>high</u> FLOPs, memory bandwidth.	<u>different</u> compute, and memory bandwidth.
Compute Utilization	<i>over-provisioned</i>	<i>tailored to compute- vs memory-heavy workload</i>
Interconnect	<i>high-bandwidth (NVIDIA NVLink, InfiniBand)</i>	<i>multiple tiers</i>
Cost Model	uniform cost	cheaper cost with schedulers
Efficiency Energy	lower.	higher.
Tool calls	<i>same GPUs</i>	CPU & high network bandwidth
LLM Prefill Phase	<i>same GPUs</i>	parallel & high compute FLOPs
LLM Decode Phase	<i>same GPUs</i>	large memory bandwidth
Asynchronous Tasks	<i>same GPUs even when waiting</i>	<i>lower-cost compute or memory-oriented nodes</i>
Scale by	adding more identical GPU racks	orchestrating heterogeneous resources

System orchestration

How workloads represented and structured?



Planning & Optimization



task $t_i \in V$ on hardware $j \in H$, consuming $\theta_{ij}^{r \in \{\text{compute, memory, bandwidth}\}}$
workloads graph $G = (V, E)$, dependency edge $(t_i, t_k) \in E$.



Goal: find an assignment that,

min. total system cost, **s.t.** latency,
capacity, and throughput constraints.



Efficient: solving optimization problem.

Scalable & flexible: easy to adapt hardware to new real-world settings.

Optimization framework

- **Goal:** find the fractional assignment $x_{ij} \in [0, 1]$ that,

$$\min_x \sum_{i \in V} \sum_{j \in H} x_{ij} \text{Cost}_{ij} + \lambda \sum_{i \in V} \text{Service Level Agreement Penalty}$$

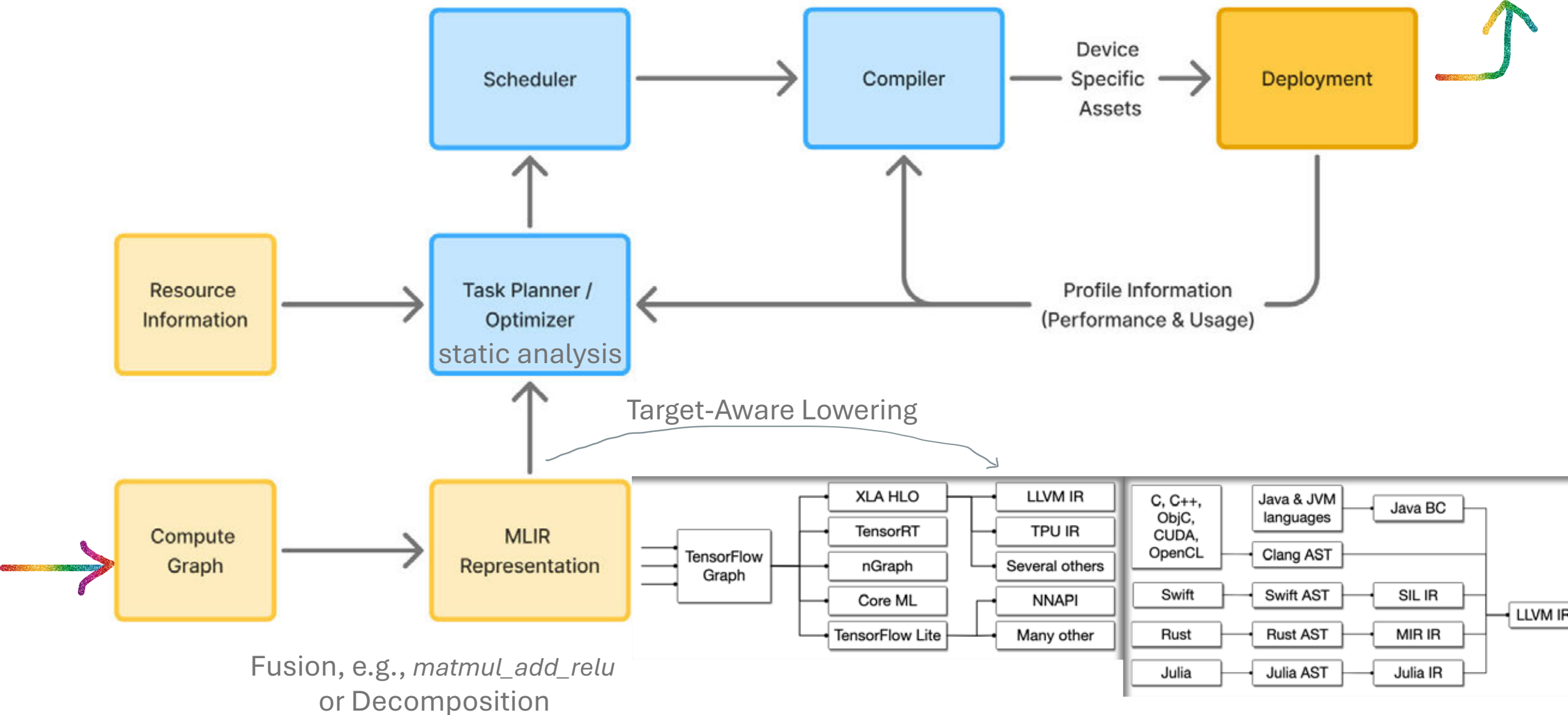
$$\text{Cost}_{ij} = \sum_r \theta_{ij}^{(r)} \cdot c_j^{(r)} + \gamma \cdot d_{ij}$$

- consumed resources $\theta_{ij}^{(r)}$ ($r \in \{\text{compute, memory, bandwidth}\}$) with per TFLOP/GB unit cost $c_j^{(r)}$.
- d_{ij} : inter-device communication cost.
- **s.t.** constraints of,
 - Resource: For each hardware, the sum of fractional assignment doesn't exceed 1.
 - The usage is below its capacity.
 - Latency: For each task, it meets the Service Level Agreement.
 - Throughput: For each task, it meets its minimum throughput requirements.
- **Solvers** from Convex optimization, trading-off performance vs. cost.

How workloads represented and structured?

- Multi-Level Intermediate Representation (MLIR)

- HW agnostic representation.
- Extensible to various AI applications, support static analysis.



How workloads represented and structured?

- Compiler lower LangChain workflow into MLIR.

(a) LangChain Orchestration (High-Level)

```
response = Agent(  
    tools=[Search(), Calculator()],  
    memory=ConversationBuffer()  
).run("Summarize the compute needs of  
      AI over the next 5 years.")
```



(b) MLIR Representation (High-Level)

```
agent.graph {  
    %0 = input.query("Summarize the  
        compute needs of AI over the  
        next 5 years.")  
    %1 = memory.load(%0)  
    %2 = llm.call(%1)  
    %3 = tool.invoke(search, %2)  
    %4 = tool.invoke(calculator, %3)  
    %5 = memory.store(%4)  
    return %5  
}
```

typed dataflow graph



(c) MLIR Representation (Decomposed + Hybrid Parallelism)

```
agent.graph {  
    %0 = input.query("Summarize the  
        compute needs of AI over the  
        next 5 years.")  
    %1 = memory.load(%0)  
  
    // Expert routing and tensor-  
        parallel prefill  
    %2 = gate.select(%1)  
    %3 = expert.tp.prefill(%1, %2)  
    %4 = merge.expert_outputs(%3)  
    %5 = allreduce(%4)  
  
    // Expert routing and tensor-  
        parallel decode  
    %6 = gate.select(%5)  
    %7 = expert.tp.decode(%5, %6)  
    %8 = merge.expert_outputs(%7)  
    %9 = allreduce(%8)  
  
    %10 = tool.lookup(search, %9)  
    %11 = tool.compute(calculator, %10)  
    %12 = memory.store(%11)  
    return %12  
}
```

exposes internal parallelism

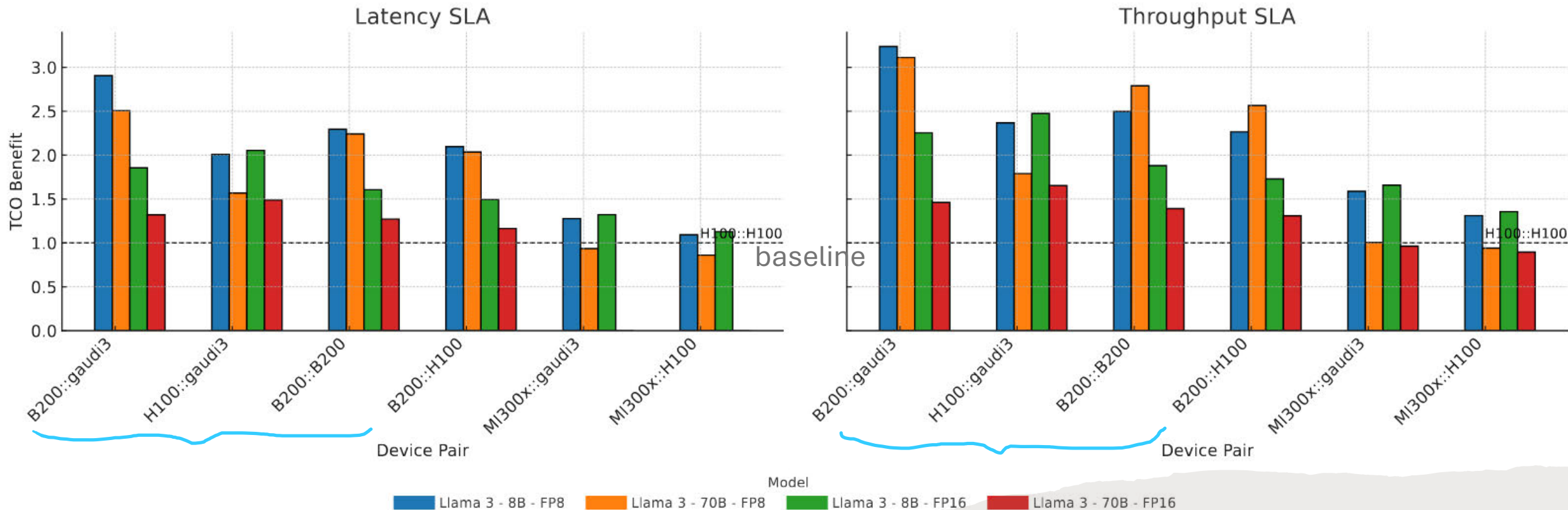
- **help** reasoning about
 - scheduling,
 - placement,
 - pipelining.
- **annotate** operations with
 - profiling metadata,
 - resource usage, or
 - placement hints.

Evaluation

- **Application:** conversational voice web LLM agent.
- Total Cost of Ownership (TCO) = Hardware + Infrastructure + Energy + Operational cost.
- *Latency:* time-to-first-token, token-to-token (ms).
- *Throughput:* tokens / (second, \$)

HW: NV B200, H100, Intel® Gaudi®
Models: Llama 3 with different FP precision, number of weights.

TCO Benefit for Heterogeneous Configs (input=4096, output=512)



Conclusions [Critique & Future Work]

- Strengths
 - system-level design (algorithms, scheduler, compiler, MLIR, HW heterogeneity).
 - workflow-aware **optimization**, based on real-time profiled data, static analysis.
 - **hardware diversity**, extending resources lifetime and lower costs.
 - Efficient and scalable design goals.
- Limitations
 - preliminary evaluation, future work for more workloads, models, scalability tests.
 - when homogeneous clusters are preferable than heterogeneous ones?
 - when overhead negate the performance gains?
- Future work
 - distributed data center, cross-device agent (cloud & edge),
 - AI Cloud Marketplace via Platform-as-a-Service (PaaS).

Thank you! Welcome for **thoughts** or **questions**~

References

- [Efficient and Scalable Agentic AI with Heterogeneous Systems](#), Stanford, 2025.
- [LLM Inference Serving: Survey of Recent Advances and Opportunities](#), B. Li et al., 2024.
 - [Attention Is All You Need](#), Google, 2017.
 - KV cache: [Efficiently Scaling Transformer Inference](#) , Google, 2022.
 - [Evaluating management of KV Cache within an inference system](#), C. Fox, 2025.
- [Convex Optimization](#), S. Boyd , Stanford, 2004.
- [MLIR: A Compiler Infrastructure for the End of Moore's Law](#), Google, 2020.